Module Global

Stage 1 status update & feedback request

Zbyszek Tenerowicz (ZTZ) @naugtur
Kris Kowal (KKL) @kriskowal

TC39 Plenary 110, 2025-09-(22–25)

Itinerary

- Problem statement review
- Motivating cases update, specifically
- The mitigation of inevitable, deeply-rooted supply chain attacks
- Feedback review
- Next steps

Problem statement review

A way to evaluate a module and its dependencies in the context of a new global scope within the same Realm

Motivating Cases

Testing

Some popular test runners create a whole realm and copy the intrinsics from the host realm into the guest in order to produce a porous emulation of compartments, effectively. With compartments, the boundary is lighter and less porous.

Safe, fast multi-tenant realms

Supply chain attack mitigation

Most websites

If a hacker gets into your datacenter and exfiltrates the unsalted hashes of all your users' passwords, security questions, and personally identifying information, your users will probably never find out and very few of them are going to leave. They lost control over all that information a long time ago, it's all for sale in a dark corner of the web, and nobody's going to trace it back to you.

The web, as it is, was made for you. However,

But you are a bank

For some applications, defending against supply chain attacks is existential.

- bank (just kidding)
- password manager
- secret store
- wallet
- most Enterprise SaaS CRUD apps (boring but profitable)
- chat (not kidding)

What do you do?

- lockfile
- integrity checks
- provenance
- ignore-scripts=true
- fund shared dependencies
- bug bounty
- audits
- lavamoat

More like inevitable

- Suppose that an attacker has succesfully obtained the right to publish arbitrary software as one of your trusted suppliers.
- Suppose they got past the malware detector after publishing.
  - Most malware detection depends on humans to spot the malware, automated detection of compromise is still new.
- Suppose they waltzed by the opportunity to move laterally in a `postinstall` script, or that you used `ignore-scripts` and `allow-scripts` to frustrate them.
- Suppose you chose the wrong moment to approve that Dependabot PR.

ansi-styles

debug

chalk

supports-color

strip-ansi

ansi-regex

wrap-ansi

color-convert

color-name

is-arrayish

slice-ansi

color

color-string

simple-swizzle

supports-hyperlinks

has-ansi

chalk-template

backslash

proto-tinker-wc

@duckdb/node-api

@duckdb/duckdb-wasm

@duckdb/node-bindings

duckdb

@coveops/abi

error-ex

In other words, over 2 billion weekly downloads

LavaMoat

You too can run malware from NPM (without consequences)

https://github.com/naugtur/running-qix-malware/

LavaMoat

1. **Trust on First Use:** Static analysis of an entire application at a snapshot in time that produces a Policy for access to powerful modules and globals, such that changes are evident and most packages are labeled as benign and of low concern.
2. **Runtime Policy Enforcement:** Enforce access to powerful globals and modules at runtime using HardenedJS `Compartment`
3. HardenedJS `lockdown` to eliminate the most severe prototype poisoning

`lockdown`, `harden`, `Compartment`

1. Lockdown freezes the "shared intrinsics" and closes some exits.
2. Harden lets you freeze an object and its transitive properties (and prototypes).
3. Compartment lets you import modules in a separate and controlled module map and in the presence of a global object that has only the shared intrinsics and additional hardened dependencies

And how?

Blocking the exits

```
Function.prototype.constructor = function () {
  throw new Error("nice try");
};

Math.random = function () {
  throw new Error("nice try");
};

Date.prototype.now = function () {
  throw new Error("nice try");
};

Number.prototype.toLocaleString = Number.prototype.toString;
// &c
```

Warning

The software you are about to see has been known to excite visceral revulsion in the viewer. Avert your gaze if you are sensitive to the use of `with`, direct `eval`, `arguments`, and `Proxy`.

```
lockdown();
assertNoImportEvalHtmlComments(suspiciousJavaScript);
const makeEvaluator = new Function(`
  with (this.scopeTerminator) {
    with (this.globalObject) {
      with (this.evalScope) {
        return function() {
          'use strict';
          return eval(arguments[0]);
        };
      }
    }
  }
`);
context.scopeTerminator = new Proxy(create(null), { has:() => true })
const evaluate = apply(makeEvaluator, context, []);
evaluate(suspiciousJavaScript);
```

But why

When we can commit these crimes today, why do we need language support for per-global module maps?

- Most JavaScript libraries work without modification.
- Biggest issue is property override mistake, and it's fading.
- CSP forces us to add `with` statements via bundling

Caveats

Imperfect emulation of strict mode.

```
export default function () {
  console.log(this);
  // this should be undefined
  // this is actually compartment.globalThis
}
```

Caveats

Imperfect emulation of strict mode.

```
shouldThrowReferenceError === undefined
```

Oddly, using a with block and an opaque scope proxy either traps all properties or traps all properties known to exist on the global object, but inadvertently reveals the shape of the host global.

Censorship

Modules must be precompiled to fit in `eval` and evade the censorship heuristics for `import`, `eval`, and HTML comments.

```
new RegExp(`(?:${'<'}!--|--${'>'})`, 'g')
new RegExp('(^|[^.]|\\.\\.\\.)\\bimport(\\s*(?:\\(|/[/*]))', 'g')
new RegExp('(^|[^.])\\beval(\\s*\\()', 'g')
```

```
/** import('@org/pkg').Type */
while (i-->0) {}
```

Language support for module maps and separate globals

- Benefit from the native module parse,
- no censorship heuristics,
- no heavy parser,
- no risk of syntax divergence,
- Faithful evaluation semantics.

Motivating cases

- Supply chain attack mitigation
- Testing infrastructure
- Safe, fast multi-tenant realms (not discussed)

Feedback review

Three problems, one solution.

1. No new paths to evaluation of text.
   - *Ergo*, we cannot rely on `eval` as the mechanism for binding dynamic `import` to a module map and globals, using execution context.
2. The name `Global` does not adequately express how new globals are distinct from The Global.
3. Do not add non-serializable hooks to `ModuleSource`.

Compartment

So, we pivot back to `new Compartment`, merging:

https://github.com/tc39/proposal-compartments.

4. Look into how this can leverage `importmap` (planned).
5. Consult implementers regarding global complications (planned).
6. Need options to avoid `importHook` trampoline for performance (addressed).

Compartment

- A place to hang an `import` method
- A name that has endured the *Shed Test*
- A place to hang undeniable intrinsics that would otherwise be ineffable without `eval`.

Resolution problem

Given:

```
// src/x.js
import "../lib/y.js";
```

That is loaded and imported as a source.

```
import source xSource from "./src/x.js";
await import(xSource);
```

The behavior here is currently host-specific and for HTML relies on the [[HostData]] internal slot.

```
// postMessage works or...
const xSource2 = structuredClone(xSource);
await import(xSource2);
```

Consider:

```
new ModuleSource(source, { base });
```

Separation of roles

Two URLs, often identical.

- moduleSource.[[HostData]] used to enforce Content-Security-Policy
- moduleSource.[[Base]] used to resolve import specifiers

```
import source a from './a/source.js';
const b = new ModuleSource(a, { base: './b/source.js' });
```

Before

```
new ModuleSource(source, {
  importHook(specifier, { with: { type } }) {
    const resolution = resolve(specifier, this.referrer);
    return import.source(resolution, { with: { type } });
  },
  referrer: import.meta.url,
})
```

Problem: Undesirable, unserializable state.

After

```
const compartment = new Compartment({
  resolveHook(specifier, referrer) {
    return import.meta.resolve(specifier, referrer);
  },
  importHook(fullSpecifier, { with: { type } }) {
    // ... returns a module handle:
    // ModuleSource | ModuleNamespace | Handle
  }
})
compartment.import(specifier);
```

```
import.source(source, { base }); // and/or
new ModuleSource(source, { base });
```

Performance: option needed to avoid hook trampoline

```
import source a from './a.js';
const compartment = new Compartment({
  modules: {
    './a.js': a,
  }
});
compartment.import('./a.js');
```

Future:

```
compartment.importNow('./a.js');
```

Handle on module record for module specifier

```
const a = new Compartment();
const b = new Compartment({
  modules: {
    'a': a.module('./src/index.js'),
  }
});
```

No new paths to evaluation of text

Before

```
// import a specified module in a new module map:
new Global().eval("specifier => import(specifier)")(specifier);
```

Alternative

```
new Global().import(specifier);
```

Better

```
const compartment = new Compartment();
compartment.import(specifier);
compartment.globalThis;
```

Evaluators

We want these, but can exclude them, or put them in an annex for non-browser implementations.

```
compartment.import(new ModuleSource('export default ()', { base }));

compartment.evaluate('42');

new compartment.globalThis.Function('return 42');

new compartment.GeneratorFunction('yield 42');

compartment.globalThis.eval('"hello"');
```

Next steps

Request out-of-band opportunity to hear from (TG3 or Module Harmony).

- Kevin Gibbons,
- Matthew Gaudet,
- Anne van Kesteren, and
- **You**

Regarding paths to evaluation, minimization of impact on HTML global categories, and your concerns.

Thank you